

HealthLock:

A Lightweight, Encryption-First Framework for Secure Patient Record Sharing

Hemant Chilkuri

B.Tech Artificial Intelligence · GH Rasoni University (GHRSTU), Nagpur, India
Beta Microsoft Learn Student Ambassador · Founder, Phoenix AI Club
hemant.chilkuri.ai@ghrstu.edu.in · github.com/CHHemant

Preprint · arXiv cs.CR · 2025

Abstract

Electronic patient record systems face a persistent tension between accessibility and confidentiality. Existing solutions either sacrifice security for usability or impose controls so restrictive they impede clinical workflows. This paper presents HealthLock, an open-source Python/Flask medical record platform resolving this tension through a layered security architecture. HealthLock implements Fernet symmetric encryption (AES-128-CBC with HMAC-SHA256) for all records at rest, role-based access control, progressive account lockout with emergency SMTP-delivered code recovery, CSRF protection, comprehensive audit logging, and cryptographically signed time-limited sharing URLs with optional QR code delivery. We describe the system architecture, a formal threat model mapped to OWASP Top 10 (2021), and a comparative evaluation against three existing systems. HealthLock is released under the MIT licence at github.com/CHHemant/HEALTHLOCK--SECURE-PATIENT-RECORD-SHARING.

Keywords: *Electronic Health Records · Fernet Encryption · Flask · Patient Privacy · CSRF · Audit Logging · Secure Sharing · Privacy-Preserving Systems*

1. Introduction

The digitisation of healthcare has produced enormous efficiency gains, yet has simultaneously expanded the attack surface for patient data. The Office for Civil Rights reported over 700 major healthcare data breaches in 2023, exposing more than 133 million records [1]. Root causes are consistent across incidents: unencrypted data stores, weak authentication, absent access controls, and no tamper-evident logging.

Clinical workflows further complicate the security picture. Clinicians routinely share records with specialists, referring physicians, and patients — a requirement that fundamentally conflicts with the principle

of minimising access. Existing solutions either sacrifice security for convenience or impose administrative overhead so heavy that clinicians route around them [2].

HealthLock proposes a third approach: a minimally viable but maximally secure record system — approximately 900 lines of Python — that makes the secure path the easy path. Every record is encrypted before it touches the database. Every share link carries a cryptographic expiry. Every authentication event is logged. The system is deployable by a single developer on commodity infrastructure, with no proprietary cloud dependency.

The contributions of this paper are:

- A reference implementation of a Flask-based EHR system with encryption-at-rest, time-limited sharing, and comprehensive audit logging.
- A formal threat model mapping 10 primary threats to mitigations and OWASP Top 10 categories.
- A comparative evaluation against three existing open-source and proprietary systems.
- A roadmap for extension toward HIPAA and GDPR technical compliance.

2. Background and Related Work

2.1 Electronic Health Record Security

Prior work on EHR security falls into three categories: cryptographic protocols for data sharing [3, 4], access control models [5], and audit and compliance frameworks [6]. Blockchain-based approaches have attracted interest for immutability properties [7], but introduce deployment complexity and latency unsuitable for routine clinical use.

Symmetric encryption with authenticated encryption schemes — particularly AES-GCM and the Python cryptography library's Fernet construction — has been identified as the appropriate baseline for record confidentiality in systems without multi-party key exchange requirements [8]. Fernet provides AES-128-CBC with PKCS7 padding and HMAC-SHA256 authentication. Its audited, self-contained design reduces implementation error risk significantly.

2.2 Capability-Based Secure Sharing

The concept of capability-based sharing — wherein a cryptographically signed token grants temporary access to a specific resource — is well-established in distributed systems literature [9]. The dangerous library implements URL-safe HMAC-signed serialisation with configurable expiry using the application's secret key, providing non-repudiation and time-boundedness without requiring a database entry per share link.

2.3 OWASP Top 10 (2021)

The OWASP Top 10 [10] provides the most widely adopted taxonomy of web application vulnerabilities. HealthLock was designed with explicit mitigations for each applicable category, with A01:2021 (Broken Access Control), A02:2021 (Cryptographic Failures), and A07:2021 (Authentication Failures) representing the three categories most critical to healthcare record systems.

3. System Architecture

3.1 Technology Stack

Component	Technology
Web framework	Flask 3.x (Python 3.10+)
ORM / Database	SQLAlchemy + SQLite (dev) / PostgreSQL (prod)
Encryption	cryptography.fernet — AES-128-CBC + HMAC-SHA256
Token signing	itsdangerous URLSafeTimedSerializer
Password hashing	Werkzeug PBKDF2-SHA256
CSRF protection	Flask-WTF (WTForms CSRF tokens)
QR generation	qrcode + Pillow
PDF generation	reportlab
Session security	HttpOnly + Secure + SameSite=Lax cookies; 60-min TTL
Deployment	Render.com (render.yaml in repository)

3.2 Data Model

The schema has three primary entities. User stores hashed credentials (PBKDF2-SHA256), failed attempt counters, lockout timestamps, emergency recovery code hashes, and registered email addresses. PatientRecord stores patient name in plaintext for indexing, record content as Fernet-encrypted binary, and an optional UUID-randomised PDF filename. AuditLog records every authentication event, record access, share link generation, and anomaly — enabling complete access history reconstruction for any record or user.

3.3 Encryption Architecture

On first startup, HealthLock generates a 256-bit Fernet key and persists it to `fernet.key`, outside the web root. This key is loaded into memory at startup for all encryption and decryption operations. The encryption flow for a new record proceeds as follows:

- The user submits a record through the authenticated web form.
- The route calls `encrypt_text(fernet, content)`, serialising plaintext to UTF-8 bytes and calling `fernet.encrypt()` to produce a URL-safe base64 token containing IV, ciphertext, and HMAC.
- The encrypted binary is stored in the `encrypted_content` column. Plaintext never touches the database.
- On retrieval, `decrypt_text(fernet, encrypted_content)` verifies the HMAC before decrypting, ensuring ciphertext integrity.

An attacker who obtains the SQLite database file without `fernet.key` obtains only authenticated ciphertext. The key file must be protected separately — ideally via environment variable injection in production — and must be excluded from version control.

3.4 Secure Sharing Subsystem

When a clinician requests a share link, the system generates a token payload containing the record ID and a Unix expiry timestamp (configurable from 1 minute to 1 week). This payload is signed with

URLSafeTimedSerializer using the application's SECRET_KEY, producing a tamper-evident, time-limited token embedded in a /share/<token> URL. When accessed, the system verifies the HMAC signature, checks the expiry, and if valid, decrypts and renders the record without requiring the recipient to authenticate — a capability-based access model appropriate for clinical handoffs.

A QR code is generated for physical display or transmission, and a separate LAN-accessible URL is provided for intranet use. The PDF share endpoint streams content inline with Content-Security-Policy: sandbox, Cache-Control: no-store, and Content-Disposition: inline, preventing caching, discouraging download, and restricting JavaScript execution in the viewer context.

4. Security Analysis

4.1 Threat Model

We consider an adversary with network access attempting: (a) credential-based attacks on authentication, (b) direct database access via SQL injection or file system compromise, (c) session-based attacks, (d) abuse of sharing functionality for unauthorised record access, and (e) cross-site attacks on authenticated users. We exclude nation-state adversaries, hardware-level attacks, and server-side root compromise.

4.2 Authentication Security

HealthLock implements progressive logout: after five consecutive failed attempts, an account locks for 300 seconds. An emergency access code (6 hex characters via secrets.token_hex) is hashed with PBKDF2-SHA256 and stored with a 10-minute expiry. If SMTP is configured, the code is delivered to the registered email, providing a recovery path without administrator intervention. Password policy requires minimum 8 characters with at least one uppercase, one lowercase, and one digit. Sessions are HttpOnly, Secure, and SameSite=Lax, with a 60-minute rolling expiry.

4.3 OWASP Top 10 Mapping

Threat	Attack Vector	HealthLock Mitigation	OWASP 2021
Credential Stuffing	Brute-force login	Lockout after 5 attempts; 5-min cooldown; emergency SMTP code	A07:2021
Session Hijacking	Stolen session token	HttpOnly+Secure+SameSite=Lax; 60-min TTL	A02:2021
Data Breach (at rest)	DB file exfiltration	Fernet AES-128-CBC+HMAC-SHA256 on all record contents	A02:2021
CSRF	Cross-site form submit	Flask-WTF CSRF tokens on all state-changing routes	A01:2021
Replay Attack	Reuse of share URL	itsdangerous time-limited signed token; configurable TTL	A01:2021
XSS	Script injection	Jinja2 auto-escape + CSP blocking inline scripts	A03:2021

Clickjacking	iFrame overlay	X-Frame-Options: DENY on all responses	A05:2021
MIME Sniffing	Content-type exploit	X-Content-Type-Options: nosniff	A05:2021
Privilege Escalation	Access other records	Owner-only enforce on every edit/delete route	A01:2021
PDF Exfiltration	Download sensitive PDF	Inline viewer; no-store cache; CSP sandbox	A02:2021

4.4 Security Response Headers

All responses include: X-Frame-Options: DENY (clickjacking prevention), Referrer-Policy: no-referrer (information leakage prevention), X-Content-Type-Options: nosniff (MIME sniffing prevention), and a Content-Security-Policy restricting scripts to self and specified CDN origins, disabling object embeds, and permitting images from HTTPS and data URIs — applied globally via an `after_request` hook.

5. Comparative Evaluation

We compare HealthLock against MedVault (proprietary cloud EHR), OpenMRS (dominant open-source EHR), and BaseVault (minimal open-source Flask record store). Table 3 presents feature comparison across security dimensions critical for OWASP-compliant implementation.

Feature	HealthLock	MedVault	OpenMRS	BaseVault
Encryption at rest	+ Fernet	+ AES-256	No	+ AES-256
Time-limited share links	+ Signed token	No	No	No
QR code access	+ Yes	No	No	No
Account lockout	+ 5 attempts	+ Yes	No	+ Yes
Emergency code recovery	+ SMTP	No	No	No
Audit logging	+ Yes	+ Yes	+ Yes	No
CSRF protection	+ Flask-WTF	+ Yes	+ Yes	No
Inline PDF viewer	+ CSP sandbox	No	No	No
Open source	+ MIT	No	+ GPL	+ MIT

HealthLock is unique among the evaluated systems in combining time-limited cryptographic share links, QR delivery, emergency account recovery, and inline PDF security — within an open-source, single-developer-deployable Python codebase, without enterprise infrastructure requirements.

6. Limitations and Future Work

6.1 Current Limitations

- Single-key encryption: All records share one Fernet key. A key compromise decrypts all records. Per-record HKDF derivation would limit blast radius.
- SQLite default: Lacks row-level locking for concurrent clinical use. PostgreSQL is supported via DATABASE_URL but Alembic migration tooling is not included.
- Single-user auth model: No multi-role RBAC. Role differentiation is required for HIPAA minimum-necessary compliance.
- Server-side decryption for sharing: True E2EE would require client-side decryption via the Web Crypto API with a key fragment in the URL hash, never sent to the server.
- No HIPAA certification: As a prototype, HealthLock makes no compliance certifications. HIPAA requires administrative, physical, and technical safeguards beyond software alone.

6.2 Future Work

- Per-patient HKDF key derivation to contain key compromise blast radius.
- Privacy-preserving RAG pipeline for semantic search over encrypted records without plaintext decryption at query time — a natural extension toward the secure NLP research agenda.
- Multi-role RBAC with Flask-Principal.
- Client-side E2EE for share links using the Web Crypto API.
- FHIR HL7 R4 API endpoint for clinical interoperability.
- Formal penetration testing against the OWASP Web Security Testing Guide checklist.

7. Conclusion

HealthLock demonstrates that a secure, production-aligned electronic health record system can be implemented in approximately 900 lines of Python by a single developer, without proprietary dependencies or cloud lock-in. By treating encryption as a first-class architectural concern and designing the sharing subsystem around cryptographic primitives, HealthLock achieves strong security properties within a developer-accessible framework.

The limitations documented in Section 6 are real and preclude direct clinical deployment without further development. As a research prototype and foundation, however, HealthLock establishes a useful baseline — what the minimum viable secure EHR looks like when built with modern Python security primitives and a principled threat model.

Complete source code, including Render.com deployment configuration, is available at github.com/CHHemant/HEALTHLOCK--SECURE-PATIENT-RECORD-SHARING under the MIT licence.

Acknowledgements

The author thanks the faculty of the Department of Artificial Intelligence, GH Rasoni University, Nagpur, for their guidance. This work was conducted as part of the author's undergraduate research programme. The author is a Beta Microsoft Learn Student Ambassador; this work is independent and does not represent the views of Microsoft Corporation.

References

- [1] U.S. Department of Health and Human Services, Office for Civil Rights. (2024). Healthcare Data Breach Statistics 2023. HHS.gov.

- [2] Kruse, C. S., et al. (2017). Cybersecurity in Healthcare: A Systematic Review of Modern Threats and Trends. *Technology and Health Care*, 25(1), 1–10.
- [3] Bendersky, M., & Hernandez, J. (2021). Symmetric vs. Asymmetric Encryption in EHR Systems. *Journal of Healthcare Informatics Research*, 5(2), 145–163.
- [4] Chen, Y., et al. (2019). A Secure Data Sharing Scheme in Cloud-Based EHR Systems. *IEEE Access*, 7, 62492–62504.
- [5] Liu, Q., et al. (2020). Role-Based Access Control for Electronic Medical Records Systems. *Journal of Medical Systems*, 44(3), 1–12.
- [6] Meingast, M., Roosta, T., & Sastry, S. (2006). Security and Privacy Issues with Health Care IT. *IEEE EMBS Annual International Conference*.
- [7] Xia, Q., et al. (2017). MeDShare: Trust-Less Medical Data Sharing via Blockchain. *IEEE Access*, 5, 14757–14767.
- [8] Percival, C., & Josefsson, S. (2016). The encrypt Password-Based Key Derivation Function. RFC 7914. IETF.
- [9] Miller, M. S., Yee, K. P., & Shapiro, J. (2003). Capability Myths Demolished. Technical Report SRL2003-02. Systems Research Laboratory.
- [10] OWASP Foundation. (2021). OWASP Top 10: 2021 Edition. owasp.org/Top10.